

FinGuard

Real-Time Financial Transaction Intelligence & AI Investigation Platform

Detailed Capstone Project Proposal

Capstone Goal

Build an end-to-end Databricks solution that processes millions of financial transactions, detects risky activity, enables AI-assisted investigations, captures operational changes with CDC, and provides analytics through a deployed Databricks App.

Primary technologies: Apache Spark | Delta Lake | Unity Catalog | Lakebase | CDC | Databricks Apps | AI/LLM | External REST API

1. Executive Summary

FinGuard is an AI-powered financial transaction monitoring and investigation platform built on Databricks. The platform ingests large volumes of financial transactions, cleans and enriches them with Apache Spark, calculates transaction risk indicators, generates investigation alerts, and provides analysts with an action-taking AI assistant.

The AI agent is designed to do more than answer questions. It can retrieve transaction and customer history, explain why an alert was generated, and perform meaningful write actions such as assigning an alert, escalating a case, adding an investigation note, updating an alert status, or resolving an investigation. Operational application data is stored in Lakebase, while Lakebase changes are replicated into Delta history tables for analytics about application usage, agent activity, and investigation performance.

The project intentionally uses the Databricks platform end to end so that the capstone demonstrates both data engineering and AI application development, rather than being only a dashboard or only a chatbot.

2. Business Scenario and Core Workflow

Financial institutions process large numbers of transactions and need efficient ways to identify potentially risky behavior and help analysts investigate alerts. FinGuard simulates this workflow using a large synthetic transaction dataset and a rules-based risk model that can later be extended with machine learning.

Example scenario: a customer normally makes small domestic purchases, but suddenly completes a high-value transaction to a new destination followed by another large international transaction within a few minutes. FinGuard calculates a high risk score, generates an alert, and presents the case to an analyst in the Databricks App.

The analyst can ask the AI agent why the alert was created, review the customer's recent transaction history, and then instruct the agent to escalate the alert and save a note for further investigation. These actions are written to Lakebase and captured by the analytics pipeline.

3. Capstone Requirement Mapping

Requirement	FinGuard Implementation
Spark data pipeline	Use Spark to ingest, clean, transform, enrich, and aggregate 1M+ financial transactions using a Bronze/Silver/Gold architecture.
Third-party API	Use Alpha Vantage CURRENCY_EXCHANGE_RATE and FX_DAILY endpoints for FX enrichment; persist raw JSON in Bronze and curated rates in Silver on defined schedules.
Lakebase data model	Store users, alerts, investigations, notes, status changes, and agent actions in relational Lakebase tables.
Action-taking AI agent	Provide read tools plus write tools for assigning, escalating, resolving, and annotating alerts.
Analytics pipeline	Capture Lakebase changes with CDC/CDF and transform them into Delta analytics tables.

Requirement	FinGuard Implementation
Frontend	Build a Databricks App with workspace authentication, role-based agent tool permissions, alert/investigation workflows, and analytics views.
Deployed application	Deploy the frontend as a Databricks App.
Big Data Vs	Demonstrate Volume (>1M rows), Variety (structured + JSON + text), and optionally Velocity with p95 transaction-to-alert latency under 60 seconds.

4. Data Sources

4.1 Financial Transaction Dataset

The primary data source will be a large synthetic financial transaction dataset such as PaySim. The dataset contains millions of transaction records, making it suitable for learning distributed Spark processing and demonstrating the Volume requirement.

Typical fields include transaction ID, customer/account identifiers, timestamp, transaction type, amount, source and destination information, account balances, and a fraud or anomaly indicator.

- Target scale: more than 1 million rows; preferably several million rows.
- Primary use: Spark ingestion, feature engineering, risk scoring, aggregations, and analytics.
- Privacy benefit: synthetic data avoids using real customer banking information.

4.2 Third-Party Financial API

FinGuard will use the Alpha Vantage Foreign Exchange APIs to enrich transactions with current and historical FX information. The initial prototype will use three configured currency pairs - USD/EUR, USD/GBP, and USD/JPY - so API volume remains predictable and demonstrable.

Example use case: when a transaction is denominated in EUR, Spark joins the latest valid EUR/USD rate from the Silver FX table, converts the amount to a standardized USD value, and uses the normalized amount during risk feature calculation. The external API is not called synchronously for each transaction; rates are cached in Delta and refreshed on schedule.

4.2.1 Endpoints, Request Shapes, and Refresh Cadence

API Function	Request Shape / Purpose	Planned Cadence	Approx. Calls/Day
Current FX rate	function=CURRENCY_EXCHANGE_RATE&from_currency={FROM}&to_currency={TO}&apikey={KEY} Current reference exchange rate.	Every 6 hours for 3 configured pairs	12
Daily FX history	function=FX_DAILY&from_symbol={FROM}&to_symbol={TO}&outputsize=compact&apikey={KEY} Historical daily OHLC FX rates.	Once daily at ~01:00 UTC for 3 pairs	3

API Function	Request Shape / Purpose	Planned Cadence	Approx. Calls/Day
Intraday FX (optional)	function=FX_INTRADAY&from_symbol={FROM}&to_symbol={TO}&interval=5min&apikey={KEY} Higher-frequency FX enrichment if premium access is available.	Not required for prototype	0

Example current-rate request:

```
GET https://www.alphavantage.co/query?
function=CURRENCY_EXCHANGE_RATE&from_currency=USD&to_currency=JPY&apikey=${ALPHA_VANTAGE_API_KEY}
```

Expected response shape:

```
{
  "Realtime Currency Exchange Rate": {
    "1. From_Currency Code": "USD",
    "3. To_Currency Code": "JPY",
    "5. Exchange Rate": "<decimal_rate>",
    "6. Last Refreshed": "<UTC_timestamp>",
    "7. Time Zone": "UTC"
  }
}
```

Example daily-history request:

```
GET https://www.alphavantage.co/query?
function=FX_DAILY&from_symbol=EUR&to_symbol=USD&outputsize=compact&apikey=${ALPHA_VANTAGE_API_KEY}
```

Expected response shape:

```
{
  "Meta Data": {"2. From Symbol": "EUR", "3. To Symbol": "USD", "6. Time Zone": "UTC"},
  "Time Series FX (Daily)": {
    "<YYYY-MM-DD>": {"1. open": "...", "2. high": "...", "3. low": "...", "4. close": "..."}
  }
}
```

4.2.2 Authentication, Throughput, Rate Limits, and Error Handling

- API authentication: store ALPHA_VANTAGE_API_KEY in a Databricks secret/App secret resource and inject it at runtime; never commit the key to notebooks, source control, logs, or frontend code.
- Expected throughput: approximately 15 successful scheduled requests per day for the three configured pairs. Calls will be serialized rather than burst concurrently, and the design stays below the standard 25-requests-per-day free-tier allowance.
- Timeouts and retries: set connection/read timeouts and retry transient failures up to three times with exponential backoff of approximately 2, 4, and 8 seconds plus jitter.
- Retryable conditions: network timeouts, HTTP 429, HTTP 5xx, or provider rate-limit messages. Invalid parameters, authentication failures, or malformed payloads are logged and not repeatedly retried.
- Fallback behavior: if a refresh fails, keep the most recent valid FX rate, record the failure in an ingestion audit record, and set an fx_is_stale/data-quality flag so downstream scoring remains available without hiding stale reference data.

4.2.3 Enrichment Table Refresh

Each successful API response is appended to `bronze_fx_api` as raw JSON with ingestion metadata. A transformation job validates and MERGEs parsed records into `silver_fx_rates`. Current-rate rows are refreshed every 6 hours and daily-history rows once per day. `silver_transactions` joins to the latest valid rate at or before the transaction event time; therefore, FX enrichment does not add an external network call to the real-time scoring path.

4.3 Application and Operational Data

User and AI-agent activity will generate operational data in Lakebase. This includes alerts, assignments, investigations, analyst notes, status changes, and AI-agent actions. These records are important because the capstone analytics requirement is about more than source transaction analytics; it must also demonstrate analytics about application usage, agent activity, or operational data changes.

5. Spark Data Pipeline and Medallion Architecture

The transaction pipeline will follow a Bronze/Silver/Gold medallion architecture. This is the core data-engineering portion of the project and should be implemented with Spark DataFrames and Spark SQL.

5.1 Bronze Layer - Raw Ingestion

- Ingest raw transaction files in batch or simulated streaming mode.
- Ingest raw third-party API JSON responses.
- Preserve original fields for traceability and replay.
- Store the data as Delta tables, for example `bronze_transactions` and `bronze_exchange_rates`.

5.2 Silver Layer - Clean and Enriched

- Remove duplicate transaction records and handle null/invalid values.
- Normalize timestamps, transaction types, currencies, and identifier formats.
- Enrich transactions with external exchange-rate or market data.
- Create customer-level rolling statistics such as average transaction amount and 30-day spend.
- Calculate transaction velocity, such as the number/value of transactions in the last 10 minutes.
- Derive flags such as `is_high_value`, `is_new_destination`, and `amount_deviation`.
- Store the result in `silver_transactions`.

5.2.1 Silver Data Quality Contracts

Silver processing will enforce explicit schema and anomaly contracts before records are used for scoring. Critical violations are quarantined for review; non-critical anomalies are retained with data-quality flags so the pipeline remains observable and reproducible.

Field / Rule	Validation / Expectation	Action on Failure
<code>transaction_id</code> / <code>customer_id</code>	Required, non-blank; <code>transaction_id</code> must be unique.	Quarantine null keys; deduplicate repeated transaction IDs.
<code>transaction_type</code>	Must belong to the configured allowed transaction-type set.	Quarantine unknown values or map known aliases during standardization.
<code>amount</code>	Numeric, finite, and > 0 .	Quarantine invalid/non-positive values; flag extreme but valid amounts for anomaly analysis.

Field / Rule	Validation / Expectation	Action on Failure
currency	Uppercase 3-character ISO-style code and present in the configured currency list.	Quarantine invalid codes; do not perform FX enrichment.
event_timestamp	Parseable and within the simulation/source time range; cannot be materially later than ingestion time.	Quarantine invalid timestamps; flag excessive lateness.
fx_rate	Numeric and > 0; source/target pair must match configured currencies.	Do not overwrite last valid rate; flag stale/missing enrichment.
schema drift	Required columns and data types must match the Silver contract.	Fail/quarantine incompatible records and record the contract violation.

- Pipeline metrics will record valid, rejected, duplicate, late, null, stale-FX, and anomalous record counts for each run/micro-batch.
- Quality checks are applied before risk scoring so HIGH/CRITICAL alerts are created only from records that satisfy the minimum scoring contract.

5.3 Gold Layer - Analytics Ready

- gold_transaction_risk - transaction-level risk results and reasons.
- gold_customer_risk - customer-level risk profile and behavioral summary.
- gold_daily_transaction_metrics - daily counts, values, and risk distributions.
- gold_alert_summary - aggregated alert and investigation metrics.
- gold_merchant_metrics - merchant/category activity and suspicious patterns.

Raw Transactions + API JSON
 ↓
 Bronze Delta Tables
 ↓ Spark cleaning / standardization
 Silver Clean & Enriched Transactions
 ↓ feature engineering / risk logic
 Gold Risk + Analytics Tables

6. Transaction Risk Scoring

For the initial capstone, FinGuard can use a transparent rules-based scoring model. This keeps the project manageable while still demonstrating Spark feature engineering and real-time decision logic. A machine-learning model can be added later as an extension.

Risk Indicator	Example Score
Transaction much larger than customer average	+30
Multiple transactions in a short time window	+20
New destination or account	+15
Unusual transaction type	+15
International transaction	+10
High-value transaction	+10

Example risk bands: 0-29 LOW, 30-59 MEDIUM, 60-79 HIGH, and 80-100 CRITICAL. Transactions at the HIGH or CRITICAL level can create investigation alerts.

7. Lakebase Operational Data Model

Lakebase will hold the transactional application state that needs fast relational reads and writes. Large historical transaction datasets remain in Delta Lake; operational workflow data belongs in Lakebase.

Table	Important Fields	Purpose
users	user_id PK, email UNIQUE, display_name, role, created_at, updated_at	Application identities, analyst roles, and ownership references.
customers	customer_id PK, customer_name, home_currency, risk_segment, status, created_at, updated_at	Operational customer context and current risk classification.
fraud_alerts	alert_id PK, transaction_id UNIQUE, customer_id FK, risk_score, risk_level, alert_reason, status, assigned_to FK, created_at, updated_at	Risk alerts generated from Gold scoring output.
investigations	investigation_id PK, alert_id FK, opened_by FK, assigned_to FK, priority, status, summary, opened_at, closed_at	Investigation cases opened for alerts.
investigation_notes	note_id PK, investigation_id FK, author_id FK, note_text, created_at	Analyst or AI-generated case notes.
agent_actions	action_id PK, alert_id FK, investigation_id FK, actor_user_id FK, agent_name, tool_name, action_type, request_payload, result_payload, action_status, created_at	Audit trail of read/write actions attempted by the AI agent.
alert_status_history	history_id PK, alert_id FK, old_status, new_status, changed_by FK, change_source, reason, changed_at	Append-only operational alert lifecycle history.

Short ERD sketch:

customers (1) ----- (*) fraud_alerts
fraud_alerts (1) ----- (*) investigations
fraud_alerts (1) ----- (*) alert_status_history
investigations (1) ----- (*) investigation_notes
investigations (1) ----- (*) agent_actions
users (1) ----- (*) fraud_alerts / investigations / notes / actions / status history

PK/FK relationships enforce alert, investigation, user, and customer referential integrity.

8. Action-Taking AI Investigation Agent

The FinGuard AI assistant will be an investigation copilot with explicit tools. Retrieval tools let the agent ground answers in project data, while write tools allow it to take meaningful actions in Lakebase.

8.1 Retrieval Tools

- `get_transaction(transaction_id)` - retrieve a specific transaction.
- `get_customer_transactions(customer_id)` - retrieve recent customer activity.
- `get_customer_risk_profile(customer_id)` - retrieve historical behavior and risk indicators.

- `get_alert(alert_id)` - retrieve alert details and current status.
- `get_investigation_notes(alert_id)` - retrieve analyst notes and case context.
- `get_exchange_rate(currency)` - retrieve external FX information when needed.

8.2 Write/Action Tools

- `assign_alert(alert_id, analyst_id)`
- `escalate_alert(alert_id)`
- `resolve_alert(alert_id, resolution)`
- `add_investigation_note(alert_id, note)`
- `update_alert_status(alert_id, status)`

8.3 Example Agent Interaction

Analyst: Why was Alert A1023 generated?

Agent: The amount is 5.4x the customer average, the destination is new, and multiple high-value transactions occurred within 10 minutes.

Analyst: Escalate this alert and add a note that the international transfer requires further review.

Agent actions: `escalate_alert("A1023")`

`add_investigation_note("A1023", "International transfer requires further review.")`

9. Analytics Pipeline Using Lakebase CDC / Change Data Feed

Changes in Lakebase operational tables will be captured and written into Delta history tables in the Lakehouse. Spark or a Delta Live Tables/Lakeflow-style pipeline will transform these change records into analytics-ready metrics.

```

Lakebase Operational Tables
fraud_alerts | investigations | investigation_notes | agent_actions
    ↓
CDC / Change Data Feed
    ↓
Delta History Tables
    ↓ Spark aggregation
Gold Application & Agent Analytics
  
```

Example analytics: alerts created per hour/day, escalations, resolution rate, average investigation duration, AI actions per day, agent action success/failure rate, most common agent actions, analyst activity, and alert status transitions.

10. Frontend - Databricks App

The frontend will be a Databricks App focused on an analyst investigation workflow. The goal is to make the project demonstrable end to end, not simply to expose notebook outputs.

- Transaction Dashboard: Transaction volume, total value, risk distribution, high-risk counts, and trend charts.

- Alert Investigation Queue: Sortable list of alerts showing customer, amount, risk score, status, and assignment.
- Alert Detail Page: Transaction details, customer history, risk factors, notes, current status, and agent recommendations.
- AI Investigator: Chat interface for explanations, history retrieval, assignments, escalation, notes, and resolution actions.
- Analytics View: Application usage, agent activity, alert lifecycle, and investigation performance metrics.

10.1 Authentication and Agent Authorization

Users will authenticate to the deployed Databricks App through the Databricks workspace OAuth/SSO flow. The app backend will use its managed service-principal identity for required platform resources. Authorization will be enforced server-side and will separate access to analytical data from operational Lakebase actions.

Role	Representative Permissions / Agent Tools
Analyst	Read alerts, transactions, customer context, and investigations; create/assign an investigation; add notes; request explanations.
Senior Analyst	All Analyst actions plus escalate alerts, change investigation status, and close/resolve alerts.
Administrator	All Senior Analyst actions plus user/role administration and controlled customer risk-profile updates.

- Unity Catalog grants will restrict Delta/Gold/SQL Warehouse access to the schemas and tables required by the app.
- Lakebase operational permissions will use PostgreSQL roles/GRANTS for the app identity; database credentials are not exposed to end users.
- Each write-capable agent tool checks the authenticated user role before execution. High-impact actions such as resolving/closing an investigation require an authorized role and explicit user intent.
- Every write action is appended to agent_actions, and alert state transitions are recorded in alert_status_history for auditability.

11. Deployment and Runtime Architecture

The Databricks App backend connects to Lakebase for operational reads/writes, Delta/SQL Warehouse for analytical queries, the AI model/agent tools for investigation assistance, and the scheduled Alpha Vantage ingestion workflow for FX enrichment. The external API is deliberately decoupled from transaction scoring so a provider delay does not block sub-minute alert generation.

11.1 End-to-End Architecture

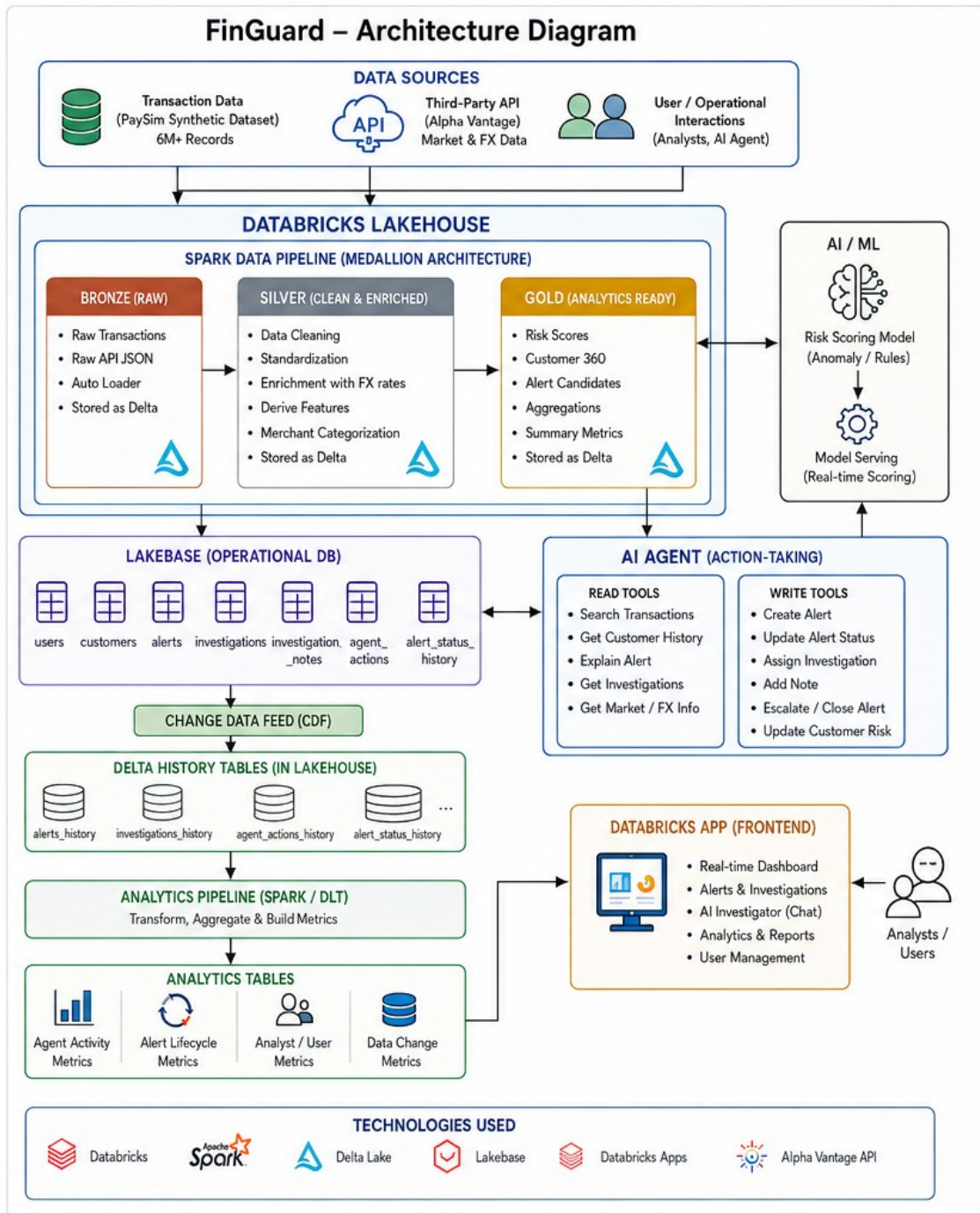


Figure 1. FinGuard end-to-end architecture.

Real-time scoring path: transaction ingest -> Bronze/Silver validation -> cached FX enrichment -> risk features -> risk score -> HIGH/CRITICAL alert upsert to Lakebase -> Databricks App/agent investigation.

12. Big Data - The 3 Vs

V	How FinGuard Demonstrates It	Capstone Status
Volume	Process more than 1 million financial transaction records with Spark; preferably several million.	Required target - Yes
Velocity	Use Spark Structured Streaming with a 10-second micro-batch trigger and target p95 transaction-arrival-to-alert-write latency below 60 seconds.	Optional credit - concrete plan included
Variety	Structured transaction data + semi-structured API JSON + unstructured notes/chat text.	Required target - Yes

FinGuard will complete Volume + Variety as the required baseline. For optional Velocity credit, the project will demonstrate a measured sub-minute transaction-to-alert path. The Velocity SLA excludes Alpha Vantage network latency because FX rates are refreshed separately and consumed from the cached Silver enrichment table.

12.1 Concrete Velocity Plan (Optional Credit)

Latency target: p95 end-to-end latency under 60 seconds from transaction arrival to persisted fraud alert. Spark Structured Streaming will use a 10-second processing-time micro-batch trigger. The streaming job reads new transactions, applies Silver validation/enrichment, calculates rolling/risk features, scores the transaction, and upserts qualifying alerts to Lakebase.

Processing Stage	Target Budget
Micro-batch wait / transaction ingestion	<= 10 sec
Bronze -> Silver validation and cached FX enrichment	<= 10 sec
Feature engineering and risk scoring	<= 15 sec
Alert-rule evaluation	<= 5 sec
Lakebase alert upsert	<= 10 sec
Network / processing overhead	<= 10 sec
Overall target	p95 < 60 sec

- Monitoring: capture input rows, processed rows, batch duration, input/processing rates, source backlog, failures, and transaction-event-time-to-alert-created-at latency.
- Validation evidence: report p50, p95, and maximum measured latency during the final demo and retain Structured Streaming progress logs/screenshots as verification.
- Idempotency: use transaction_id/alert_id uniqueness and MERGE/upsert logic so retrying a micro-batch does not create duplicate alerts.

13. Databricks Technologies and Learning Outcomes

Technology	What You Learn / Build
Apache Spark / Spark SQL	Distributed cleansing, enrichment, feature engineering, aggregations, and large-scale processing.

Technology	What You Learn / Build
Spark Structured Streaming	Optional Velocity path using a 10-second micro-batch trigger, stateful rolling features, sub-minute risk scoring, and measurable transaction-to-alert latency.
Delta Lake	Reliable Bronze/Silver/Gold storage and analytical history tables.
Unity Catalog	Organize, govern, and control access to Lakehouse datasets.
Lakebase	Operational relational storage for alerts, investigations, notes, and AI actions.
CDC / Change Data Feed	Capture operational changes and feed downstream analytics.
Databricks Workflows	Orchestrate batch/streaming pipelines, API ingestion, scoring, and analytics jobs.
Databricks SQL	Query Gold tables and support dashboard/analytics workloads.
Databricks Apps	Deploy the analyst-facing application.
AI/LLM + tool calling	Build an agent that retrieves grounded information and performs controlled write actions.

14. Recommended Implementation Phases

1. Phase 1 - Spark + Delta: ingest the large transaction dataset and build Bronze, Silver, and Gold tables.
2. Phase 2 - Unity Catalog: organize project schemas/tables and establish clean naming/governance.
3. Phase 3 - Risk Scoring: implement transparent rule-based risk features and alert candidates.
4. Phase 4 - Third-Party API: ingest FX/market JSON and enrich Silver transactions.
5. Phase 5 - Lakebase: create operational tables for alerts, investigations, notes, and agent actions.
6. Phase 6 - CDC Analytics: capture Lakebase changes into Delta and build application/agent metrics.
7. Phase 7 - AI Agent: add retrieval tools first, then controlled write tools.
8. Phase 8 - Databricks App: build dashboard, alert queue, investigation page, and AI interface.
9. Phase 9 - Workflows & Deployment: orchestrate pipelines and deploy the completed app.
10. Phase 10 - Optional Velocity Validation: replay transactions with a 10-second Structured Streaming micro-batch, measure p50/p95/max transaction-to-alert latency, and document monitoring evidence.

15. Expected Final Demonstration

1. Ingest and process millions of financial transactions with Spark.
2. Show Bronze, Silver, and Gold Delta tables.
3. Demonstrate enrichment from the third-party financial API.
4. Show transaction risk features and risk scores.
5. Generate a high-risk/critical alert.
6. Open the alert in the Databricks App.
7. Ask the AI agent why the transaction was flagged.
8. Ask the agent to escalate the alert and add an investigation note.

9. Show the corresponding records written to Lakebase.
10. Show CDC capturing the Lakebase changes into Delta history tables.
11. Display analytics for alert lifecycle, analyst activity, and AI-agent actions.

16. Success Criteria

- Spark successfully processes more than 1 million transaction rows.
- At least one external financial API is integrated and its data is used in the project.
- Lakebase contains a normalized operational data model with successful application writes.
- The AI agent demonstrates both retrieval and at least one meaningful write action.
- Lakebase changes are captured into Delta history/analytics tables.
- The Databricks App supports the core investigation workflow and is deployed successfully.
- The project clearly demonstrates at least two Big Data Vs: Volume and Variety.
- The final demo shows the complete flow from transaction ingestion to investigation analytics.
- If Velocity credit is claimed, demonstrate p95 transaction-arrival-to-alert-write latency below 60 seconds with retained streaming progress/latency evidence.

17. Implementation Clarifications for Required Revisions

The following implementation choices clarify the frontend, CDC, streaming operations, physical data design, and AI-agent plan for the FinGuard capstone.

17.1 Frontend Stack and Service Layer

FinGuard will use Streamlit with Python for the Databricks App frontend. The browser will communicate with a Python service layer and will not connect directly to Lakebase or Delta tables. Backend modules will include `agent_service.py`, `alert_service.py`, `investigation_service.py`, and `authorization_service.py`.

Lakebase reads and writes will be executed server-side with `psycopg` using the Databricks App service-principal identity. Delta and Gold analytics will be queried through Databricks SQL with Unity Catalog grants. Write-capable tools such as `assign_alert`, `escalate_alert`, `resolve_alert`, `add_investigation_note`, and `update_alert_status` will validate the authenticated user, role, request parameters, and current case state before execution.

17.2 Lakebase-to-Delta CDC Setup

FinGuard will use native Lakebase Change Data Feed (CDF) as the preferred CDC path, subject to workspace availability. Operational tables including `fraud_alerts`, `investigations`, `investigation_notes`, `agent_actions`, and `alert_status_history` will be configured with `REPLICA IDENTITY FULL` so update events can capture complete before-and-after row information.

CDF records will be written to a Unity Catalog schema such as `finguard.lakebase_cdc` using history tables including `lb_fraud_alerts_history`, `lb_investigations_history`, `lb_investigation_notes_history`, `lb_agent_actions_history`, and `lb_alert_status_history_history`. CDF capture will run continuously, and an incremental Spark/Lakeflow analytics task will process newly captured changes approximately every 1 minute. If native Lakebase CDF is unavailable, Lakeflow Connect PostgreSQL CDC will be used as the fallback.

17.3 Streaming Runtime and Operations

The optional Velocity path will use Spark Structured Streaming with a 10-second processing-time trigger on dedicated Databricks Jobs compute. The initial configuration will use one driver and two fixed workers with autoscaling disabled for predictable streaming behavior. If p95 latency testing exceeds the under-60-second target, the stream will be scaled to four fixed workers.

Each stream will use an independent Unity Catalog Volume checkpoint location, such as `/Volumes/finguard/operations/checkpoints/transaction_risk_v1`. Transaction-velocity calculations will use a 10-minute risk window and a 30-minute event-time watermark. Stateful workloads will use the RocksDB state store where available.

Idempotency will be enforced with `transaction_id` as the business key. Gold risk writes will use Delta MERGE, while Lakebase fraud_alerts will enforce a `UNIQUE(transaction_id)` constraint and use `INSERT ... ON CONFLICT DO UPDATE` so replayed micro-batches cannot create duplicate alerts.

17.4 Delta Layout and Lakebase Indexing

For new Delta tables, FinGuard will use liquid clustering where supported. Planned clustering keys are `event_date` for `bronze_transactions`; `event_date` and `customer_id` for `silver_transactions` and `gold_transaction_risk`; `customer_id` for `gold_customer_risk`; and `event_date` for daily and alert summary tables. If liquid clustering is unavailable, large transaction tables will be partitioned by `event_date` and optimized on common filter columns such as `customer_id`.

Lakebase will use primary-key indexes plus targeted secondary indexes on `fraud_alerts(transaction_id)`, `fraud_alerts(customer_id)`, `fraud_alerts(status, created_at)`, `fraud_alerts(assigned_to, status)`, `investigations(alert_id)`, `investigation_notes(investigation_id, created_at)`, `agent_actions(alert_id, created_at)`, and `alert_status_history(alert_id, changed_at)`.

17.5 AI Model, Tool Calling, and Guardrails

FinGuard will use a Databricks-hosted instruction model through Foundation Model APIs and Unity AI Gateway. The initial model plan is Meta Llama 3.3 70B Instruct, subject to availability in the course workspace. The agent will use an OpenAI-compatible function/tool-calling pattern in which the model may request an approved function but FinGuard application code performs the actual call after validation.

The agent will not receive an arbitrary SQL execution tool. Only predefined allowlisted tools will be exposed. Tool arguments will be schema-validated, authenticated user roles will be checked server-side, current alert/investigation state will be validated, and high-impact actions such as escalation or resolution will require confirmation. Every write action will be recorded in `agent_actions` with actor, model/agent, tool name, parameters, affected record, execution status, and timestamp.

18. Final Project Summary

FinGuard is designed to be a strong Databricks learning project because every required component has a clear role in the business workflow. Spark and Delta handle high-volume transaction engineering, the external API enriches financial context, Lakebase manages operational investigation

state, CDC connects application changes to analytics, the AI agent provides grounded assistance and actions, and Databricks Apps provides a usable deployed interface.

The result is a complete end-to-end capstone: data ingestion -> transformation -> risk detection -> operational workflow -> AI-assisted action -> CDC analytics -> deployed application.

Recommended scope: complete Volume + Variety first. Then enable the documented 10-second micro-batch Velocity path and validate p95 transaction-to-alert latency under 60 seconds if optional Velocity credit is pursued.